Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre

A Project Report on

"2D Snake Game"

[Code No: COMP 202]

(For partial fulfillment of Year II / Semester I in Computer Engineering)

Submitted by

Kunal Panijyar (67)

Submitted to

Er. Sagar dhakal
Department of Computer Science and Engineering

February 24, 2026

This project work on

"2D Snake Game"

is the bonafide work of

"Kunal Panijyar"

Acknowledgement

I would like to express my sincere appreciation to my Data Structures and Algorithms instructor for assigning this project and providing the opportunity to apply theoretical concepts in a practical manner. This project helped me build my understanding of problem-solving techniques and algorithmic thinking.

I am also grateful for the academic environment that encouraged me for independent learning in the development of this project.

It helped me to learn and improve my skills in logical thinking, problem solving, and program design. It provided me good experience and add confidence in implementing data structures concepts in real-world applications.

Abstract

The Snake Game is a classic arcade game where the player controls a snake to eat food while avoiding collisions with walls or itself. This project implements the game using **C++** and the **Raylib graphics library**, demonstrating practical application of data structures, real-time event handling, and graphical rendering.

The snake grows with every piece of food consumed, and the game tracks the player's score. Collision detection ensures the game resets appropriately upon hitting walls or the snake's own body. This project provides a hands-on experience in game programming, using **deques** for the snake's body, vector math for movement, and integrating audio for feedback.

Keywords: Snake Game, C++, Raylib, Deque, Game Programming, Collision Detection

Table of Contents

Acknowledgement 1
Abstract 2
Table of Contents 3
Chapter 1: 6
Introduction 6
1.1 Background 6
1.2 Objectives 6
1.3 Scope of the project 6
Chapter 2: 7
System Overview 7
2.1 Game Description 7
2.2 Features of the Game 7
2.3 Rules and Gameplay 7
2.4 Technologies used 8
2.5 Drawbacks 8
Chapter 3: 9
Methodology 9
3.1 Algorithm used 9
3.2 Data Structures Used 9
3.3 Program Flow 9
Chapter 4: 10
Implementation 10
4.1 Tools and Technologies Used 10
4.2 Code Structure 10
4.3 Key function explanation 11
Chapter 5: 11
Result and Discussion 11
5.1 Output Screens 11
5.2 Performance and Testing 13
5.3 User Interaction 14
Conclusion 15

Future Improvements 16
Reference 17

Table of Figures

Figure 1: Initial output of the snake game 12
Figure 2: Output of Snake moving and eating food. 13

Chapter 1:

Introduction

1.1 Background

As we know that computer games are a fun and effective way to learn programming and improve problem-solving skills. One of the most popular classic games is the Snake Game. The game first appeared in arcade machines in the late 1970s, but it became world-famous in the late 1990s when it was pre-installed on Nokia mobile phones. Because these phones were widely used, millions of people played Snake, making it one of the most recognized mobile games of all time.

The gameplay is simple: the player controls a snake that moves around the screen, eats food, and grows longer. The challenge is to avoid hitting the walls or the snake's own body. Despite its simplicity, the game requires quick thinking and coordination.

This project focuses on developing the Snake Game using **C++** and the Raylib graphics library. The game displays important programming concepts such as loops, conditions, arrays, and real-time input handling while providing an interactive experience.

1.2 Objectives

The primary objectives of this project include:

1. To design and develop a simple Snake Game using C++.
2. To understand and apply basic data structure concepts in game development.
3. To implement real-time movement and collision detection.
4. To create an interactive and user-friendly gaming experience.
5. To enhance problem-solving and logical thinking skills through practical implementation

1.3 Scope of the project

This project focuses on creating a single player Snake Game that runs on a computer. The game has basic features such as controlling the snake's movement, generating food, tracking the score, and ending the game when the snake hits the wall or itself.

The main purpose of this project is to understand programming logic and apply data structure concepts, rather than building a complex commercial game. As, the current version is simple, it provides a strong foundation for learning game development. In the future, additional features such as multiple levels, sound effects, and improved graphics can be added to make the game more engaging.

Chapter 2:

System Overview

2.1 Game Description

The Snake Game is a simple arcade-style game in which the player controls a snake that moves around the screen. The objective is to eat the food that appears randomly. Each time the snake eats the food, it grows longer and the player's score increases.

The game is developed using **C++** and the Raylib library, which provides graphics and real-time interaction. The player controls the snake using the keyboard, making the game interactive and easy to play.

2.2 Features of the Game

The game includes several basic but important features:

1. Smooth movement of the snake
2. Keyboard controls for direction changes
3. Random generation of food
4. Snake grows longer after eating food
5. Score tracking system
6. Collision detection
7. Game-over message when the snake hits the wall or itself
8. Simple and user-friendly interface

2.3 Rules and Gameplay

The game follows simple rules:

1. The snake moves continuously on the screen.
2. The player controls the snake using arrow keys.
3. Food appears randomly on the screen.
4. Eating food increases the score and length of the snake.
5. The snake must avoid hitting the walls or its own body.
6. The game ends if a collision occurs.

The goal is to survive as long as possible and achieve a high score.

2.4 Technologies used

The following technologies were used to develop this project:

1. C++ Programming Language – used to implement the game logic and structure.
2. Raylib Graphics Library – used for rendering graphics, handling input, and creating smooth animation.
3. Keyboard Input Handling – used to control the snake's direction.

These technologies make the game lightweight, efficient, and suitable for learning game development basics

2.5 Drawbacks

Although the game works properly, it has some limitations:

1. The game has only one level and difficulty setting.
2. Sound effects and background music are not included.
3. Graphics are simple and not highly detailed.
4. There is no pause or restart button within the game window.
5. High scores are not saved after closing the game.

These limitations provide opportunities for future improvement.

Chapter 3:

Methodology

3.1 Algorithm used

The Snake Game implemented in this project follows a classic snake movement and growth algorithm. The game maintains the snake as a dynamic sequence of segments represented using a "deque". At every update interval, the snake moves in the current direction by adding a new segment at the head. If the snake eats food, the tail is not removed, effectively increasing the length of the snake. Collision detection is used to handle interactions with walls and self-collision. The game loop continuously updates the snake's position and redraws the game board until the window is closed. Keyboard input controls the snake's direction, and the update interval ensures smooth and consistent movement.

3.2 Data Structures Used

The primary data structures used in this implementation include:

1. **Deque (std::deque)**: Used to store the snake's body segments in order. This allows efficient addition and removal of segments from both ends.
2. **Vector2 (Vector2)**: Represents coordinates of snake segments and food on the grid. It stores x and y positions as floating-point numbers.
3. **Boolean flags (bool)**: Used for controlling game states such as movement allowance (allowMove) and growth (addSegment).
4. **Color and Texture objects**: Used to define the visual appearance of the snake and food.

This combination of structures provides efficient manipulation of the snake's movement and easy checking for collisions.

3.3 Program Flow

The program follows a structured flow:

1. **Initialization**: The game window, audio device, and game objects (snake and food) are initialized.
2. **Main Game Loop**: The loop continues until the window is closed. Inside the loop:

    i. Input is read to update the snake's direction.

ii. The game state is updated at fixed intervals, including snake movement and collision checks.
iii. The game board is drawn, including snake, food, score, and boundaries.

3. **Collision Handling**: The game checks for collisions with food, walls, and the snake's own tail. If a collision occurs, appropriate actions like resetting the snake or incrementing score are taken.
4. **Termination**: On window close, resources like textures and sounds are unloaded and the program exits gracefully.

Chapter 4:

Implementation

4.1 Tools and Technologies Used

The Snake Game is developed using **C++**, which is a widely used programming language known for its efficiency and control over system resources. C++ allows the game to run smoothly and handle dynamic data, such as the snake's growing body, effectively. To create the game's visuals, **Raylib** is used, which is a simple and beginner-friendly graphics library designed for 2D game development. Raylib makes it easy to draw the snake, food, game boundaries, and score on the screen while maintaining smooth animations. The game also incorporates sound effects, such as eating food and collisions, using **Raylib's audio module**. This module enables the program to load and play audio files seamlessly, enhancing the overall gaming experience. The development environment can be any **C++ IDE** that supports C++11 or higher, allowing easy compilation and execution of the program. The game is cross-platform, meaning it can be played on **Windows** without any significant changes, making it accessible to a wide range of users. Overall, the combination of C++, Raylib, and simple development tools ensures that the game is interactive, visually appealing, and easy to run on different systems.

4.2 Code Structure

The project is organized in an object-oriented manner:

1. **Snake Class**: Handles the snake's body segments, movement, drawing, and reset functionality.
2. **Food Class**: Manages food placement, random generation, and drawing on the screen.
3. **Game Class**: Integrates snake and food objects, handles scoring, collisions, audio effects, and overall game state.

4. **Main Function**: Initializes the game, handles input, and maintains the game loop including drawing and updates.

This modular structure allows easy maintenance and potential expansion of game features.

4.3 Key function explanation

1. **ElementInDeque(Vector2, deque<Vector2>)**: Checks if a given coordinate already exists in the snake body, used for preventing food spawn inside the snake.
2. **EventTriggered(double)**: Controls the game update interval, ensuring smooth movement independent of frame rate.
3. **Snake::Update()**: Moves the snake by adding a new head segment and removing the tail unless the snake has eaten food.
4. **Game::CheckCollisionWithFood()**: Detects if the snake eats food, updates the score, grows the snake, and plays a sound.
5. **Game::CheckCollisionWithEdges() & Game::CheckCollisionWithTail()**: Detect collisions that result in a game over.
6. **Food::GenerateRandomPos(deque<Vector2>)**: Ensures new food is spawned in a free cell, not on the snake body.

Chapter 5:

Result and Discussion

5.1 Output Screens

When the game runs, it shows a rectangular playfield that is divided by grid lines. These lines help the player see the movement of the snake clearly and understand where it is on the board. The snake itself is colored in dark green and moves smoothly across the screen. As the snake eats food, it grows longer, which makes the game more challenging and fun.

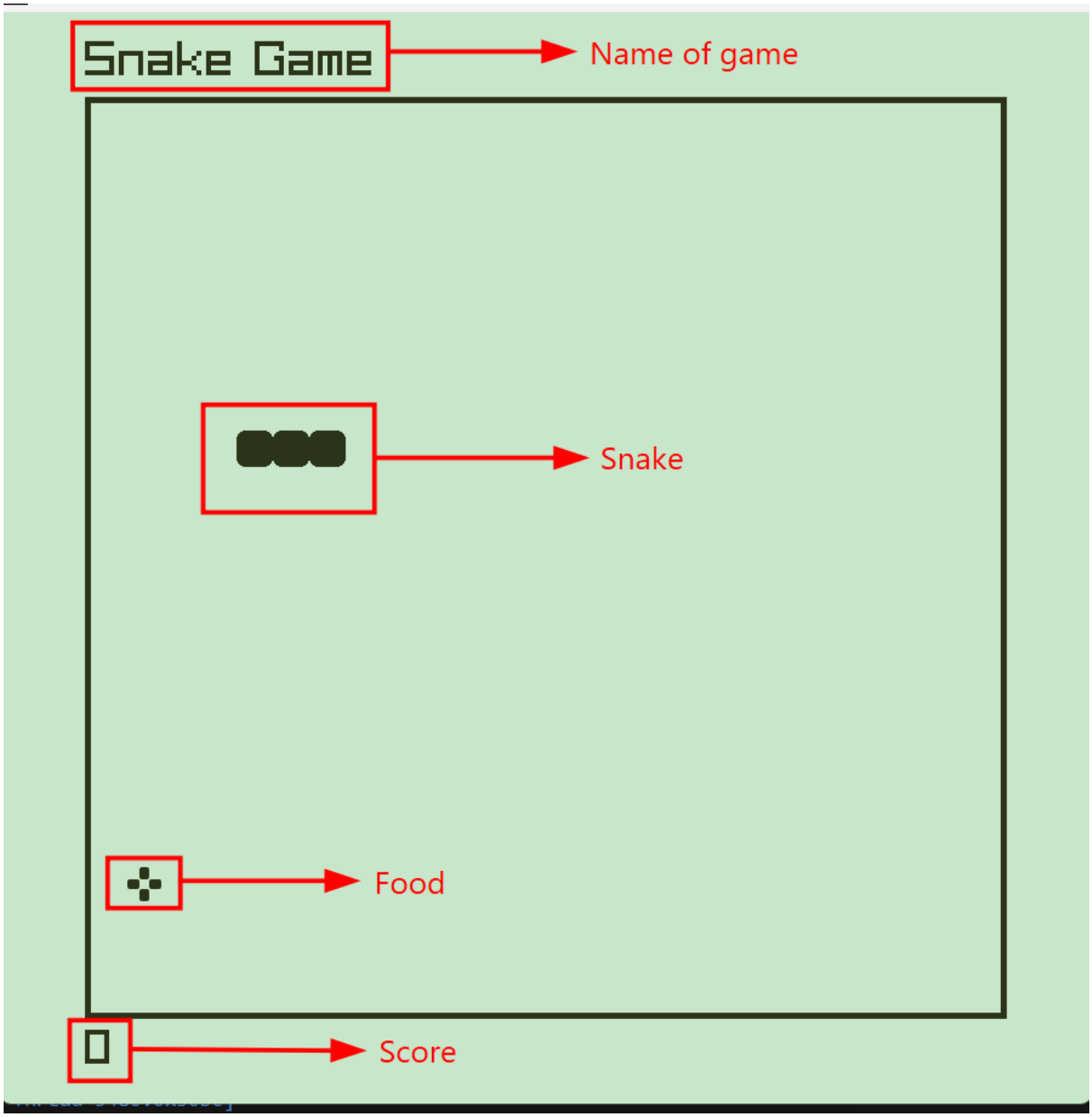

Figure 1: Initial output of the snake game

The food is displayed using an image texture, making it visually distinct from the snake and the background. Every time the snake eats the food, a new piece appears at a random position on the playfield, keeping the game dynamic. The current score is shown below the playfield, so the player can keep track of their progress easily. Additionally, the game title is displayed at the top of the window, giving the game a complete and organized look.

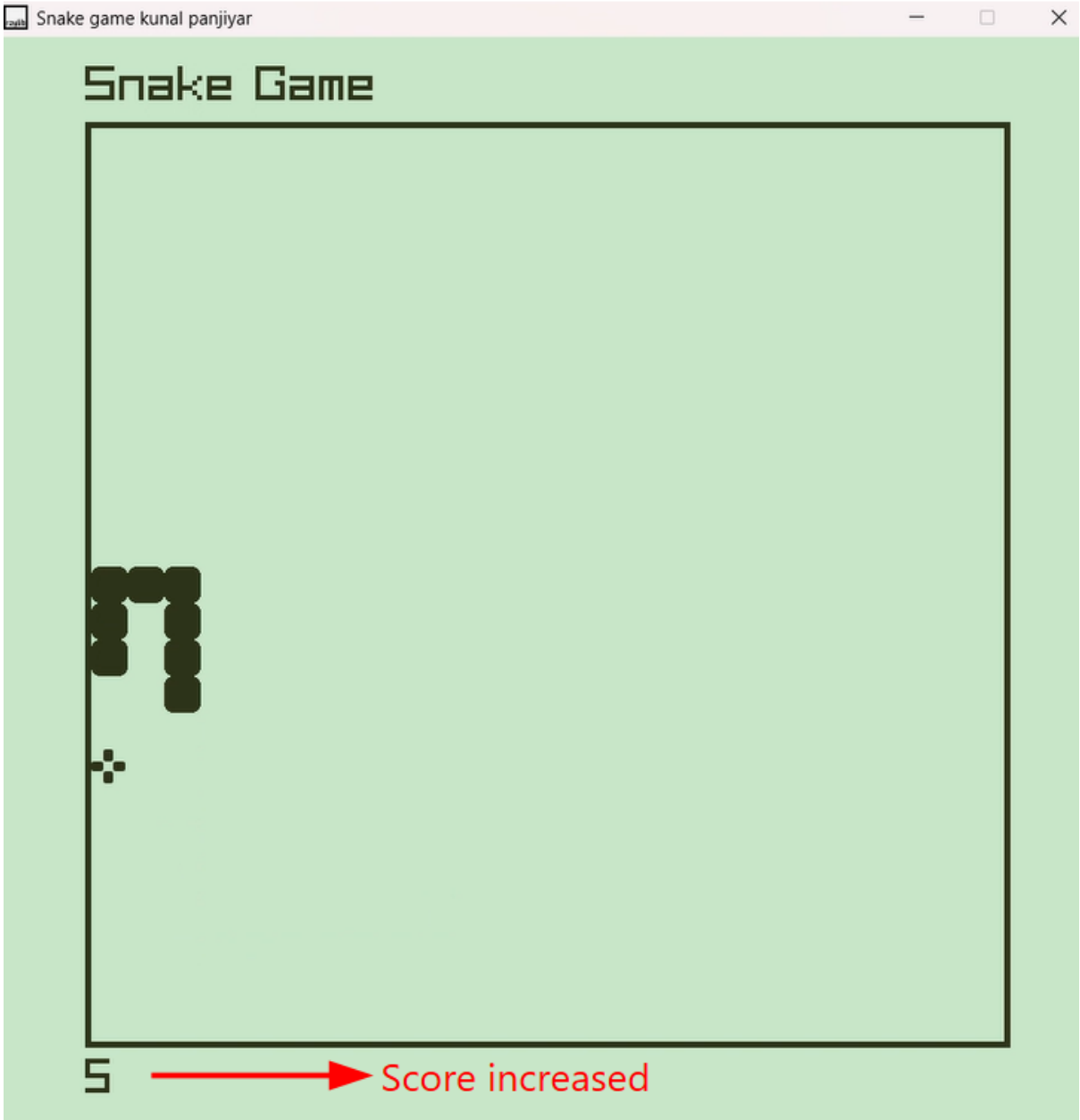

Figure 2: Output of Snake moving and eating food.

Overall, the game is interactive and responsive. The combination of clear colors for the snake, food, and background, along with smooth movement and sound effects, makes it easy to play and visually appealing. Players can immediately understand the game mechanics and enjoy the experience without confusion.

5.2 Performance and Testing

The Snake Game performs very smoothly, running consistently at 60 frames per second (FPS), which ensures that the animations and movements appear fluid and natural. The snake's movement is updated every 0.2 seconds, allowing the player to react easily while keeping the game challenging. Extensive testing was done to make sure that all collisions are detected correctly. This includes collisions with the walls, the snake hitting its own tail, and the snake

successfully eating the food. Each of these cases triggers the expected behavior, such as ending the game or increasing the score.

The game also includes audio feedback, which makes the experience more engaging. There are distinct sounds for when the snake eats food and when it collides with a wall, giving the player clear signals about what is happening in the game. Despite the snake growing longer over time, the game maintains stable performance because it uses the deque data structure efficiently. This allows the program to add or remove segments from the snake's body quickly without slowing down, even during longer gameplay. Overall, the game has been tested thoroughly and provides smooth, reliable, and responsive performance throughout.

5.3 User Interaction

The Snake Game is smooth and responsive, with instant reaction to player inputs via arrow keys. Collision detection for walls, the snake's tail, and food works correctly, and audio feedback enhances the gameplay experience. Performance remains stable even as the snake grows, with no noticeable slowdowns or crashes. Overall, the game is interactive, reliable, and enjoyable for the player.

Conclusion

Lastly, the Snake Game project successfully demonstrates how to build an interactive and fun 2D game using C++ and Raylib. The game shows the effective use of programming concepts like classes, functions, and data structures, especially the deque for managing the snake's body. The player can control the snake smoothly, collect food, and watch the score increase, making the game engaging. Collision detection works correctly, and the game responds instantly to user input, providing a reliable and enjoyable experience. The combination of clear visuals, sound effects, and smooth movement makes the game easy to play and visually appealing. Overall, the project achieves its goal of creating a simple, interactive, and well-functioning snake game while demonstrating important programming and game development concepts.

Future Improvements

1. **Implement levels with increasing speed**: The game can be made more challenging by introducing different levels. As the player progresses, the snake could move faster, making it harder to control and adding more excitement to the gameplay.
2. **Add power-ups or special food items**: Special items could appear occasionally on the playfield that give the player extra points, temporary speed boosts, or other abilities. This would make the game more interesting and fun.
3. **Save high scores to a file**: Currently, the score resets when the game ends. By saving high scores to a file, players can track their best scores over time, which adds a sense of achievement and replay value.
4. **Implement multiplayer mode or AI-controlled snakes**: The game could allow two players to play together or include computer-controlled snakes as opponents. This would make the game more interactive and competitive.
5. **Enhance graphics with animations and effects**: Adding animations, smooth transitions, and visual effects for eating food or collisions would make the game look more lively and appealing to the player.

Reference

Interactive, N. (23rd February, 2023). *Snake*. Snake. https://playsnake.org/#google_vignette

Udacity. (5th January,2021). *GitHub - udacity/CppND-Capstone-Snake-Game: A 2D Snake game using C++ and SDL*. GitHub. https://github.com/udacity/CppND-Capstone-Snake-Game

Educ8s. (n.d.). *GitHub - educ8s/Cpp-Retro-Snake-Game-with-raylib: C++ Retro Snake Game with raylib with a video tutorial explaining each line.* GitHub. https://github.com/educ8s/Cpp-Retro-Snake-Game-with-raylib

Ronan-G. (n.d.). *GitHub - Ronan-G3/CPP-RaylibSnakeGame: My first project in C++. Used Raylib to create the game Snake! A childhood favorite of mine. . .. well at least my favorite to play on the Nokia 3210 ;)*. GitHub. https://github.com/Ronan-G3/CPP-RaylibSnakeGame